

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 14

дисциплина: Архитектура компьютера

Студент: Ромеро Гаспар Фернандо Алексей

Группа: НКАбд-02-25

МОСКВА

2026 г.

Оглавление

1. Цель работы
2. Задание и последовательность выполнения работы
 - 2.1. Скрипт с семафором и синхронизацией процессов
 - 2.2. Скрипт для имитации команды `map`
 - 2.3. Скрипт для генерации случайных последовательностей букв
3. Выводы
4. Список литературы

Цель работы

Целью данной лабораторной работы является изучение основ программирования в оболочке ОС UNIX и получение навыков написания более сложных командных файлов с использованием логических управляющих конструкций и циклов, а также освоение работы с семафорами, генерацией случайных чисел и созданием аналогов системных команд.

Задание и последовательность выполнения работы

В этом разделе представлены задания, которые необходимо выполнить, и подробные инструкции по созданию скриптов на языке `bash` для реализации семафоров, имитации команды `map` и генерации случайных последовательностей букв.

2.1. Скрипт с семафором и синхронизацией процессов

Задание: Написать командный файл, реализующий упрощённый механизм семафоров. Командный файл должен в течение некоторого времени t_1 дожидаться освобождения ресурса, выдавая об этом сообщение, а дождавшись его освобождения, использовать его в течение некоторого времени $t_2 < t_1$, также выдавая информацию о том, что ресурс используется. Запустить командный файл в одном виртуальном терминале в фоновом режиме, перенаправив его вывод в другой терминал. Доработать программу для взаимодействия трёх и более процессов.

Теоретическая справка: Семафор — это механизм синхронизации, который контролирует доступ к общему ресурсу в многозадачных системах. В Bash семафор можно реализовать с помощью файлов-блокировок, где наличие файла указывает на занятость ресурса.

Шаг 1: Создание скрипта семафора

```
alexexi@fedora:~/work/os/lab14$ nano semaphore.sh
```

```
alexexi@fedora:~/work/os/lab14$ chmod +x semaphore.sh
```

Код скрипта (для двух процессов):

```

GNU nano 8.7.1 semaphore.sh
#!/bin/bash
#script: semaphore.sh
#description: Implements a semaphore mechanism for process synchronization
#Time settings (in seconds)

T1=15 #Resource waiting time
T2=8 #Resource usage time (T2 < T1)
#Lock file (semaphore)
SEMAPHORE_FILE="/tmp/semaphore.lock"
#Get the terminal number where the output (argument) id redirected to
if [ $# -ne 1 ]; then
echo "Usage: $0 <terminal number>"
echo "Example: $0 2 #Fore redirection to /dev/tty2"
exit 1
fi
TERMINAL="/dev/tty$1"
PID=$$
#Function for writing messages to the target terminal
log_message (){
echo "[$(DATE '+%H:%M:%S')] Process $PID: $1" > "$TERMINAL"
}
log_message "Starting the process. Standby time: ${T1}c, Usage time: ${T2}c"
log_message "An attmpt to get a resource"
#An attempt to ovtain a resource (semaphore)
WAIT_TIME=0
while [ $WAIT_TIME -lt $T1 ]; do
if [ ! -f "$SEMAPHORE_FILE" ]; then
#Resource release: create a lock file
echo "$PID" > "$SEMAPHORE_FILE" ]; then
log_message "Resource received (semaphore created)"
#Resource usage within T2 seconds
log_message "Resource usage... (${T2}c)"
sleep $T2
#Resource release
rm -f "$SEMAPHORE_FILE"
log_message "Resource released (semaphore deleted)"
exit 0
else
#Resource occupation:waiting
WAIT_TIME=$((WAIT_TIME + 1))
log_message "The resource is occupied by the process $(cat $SEMAPHORE_FILE 2>/dev/nul>
sleep 1
fi
done
log_message "Time out (${T1} c). Couldn't get the resource"
exit 1

```

Шаг 2: Запустите скрипт в двух терминалах

Терминал 1 (основной):

```

alexei@fedora:~/work/os/lab14$ ./semaphore.sh 2
alexei@fedora:~/work/os/lab14$ cat /dev/tty2

```

Шаг 3: Версия для трех процессов (semaphore_multiple.sh)

```

alexei@fedora:~/work/os/lab14$ nano semaphore_multiple.sh
alexei@fedora:~/work/os/lab14$ chmod +x semaphore_multiple.sh

```

Код скрипта (для нескольких процессов):

```

GNU nano 8.7.1 semaphore_multiple.sh
#!/bin/bash
#script: semaphore_multiple.sh
#Description: Implements a semaphore fore multiple process(three or more)
#Maximum number of processes that can use the resource simultaneously

MAX_PROCESSES=1
T1=15
T2=5
#Directory for locking files

SEMAPHORE_DIR="/tmp/semaphore_dir"
#Create the directory if it doesn't exist
mkdir -p "$SEMAPHORE_DIR"
#Get the terminal number where output is redirected
if [ $# -lt 1 ]; then
echo "Usage: $0 <terminal_number> [process_id]"
echo "Example: $0 2 1"
exit 1
fi
TERMINAL="/dev/tty$1"
PROCESS_ID="$2:-$$"
PID=$$
LOG_MESSAGE(){
echo "[$(date '+H:%M:%S')] Process $PROCESS_ID: $1" > "$TERMINAL"
}
log_message "Starting process. Waiting time: ${T1}s, usage: ${T2}s"
log_message "Attempt to acquire resource..."
#Function for counting actives processes
#Function for attempting to acquire a slot
acquire_slot(){
local slot
for slot in $(seq 0 $((MAX_PROCESSES - 1))); do
if [ ! -f "$SEMAPHORE_DIR/slot_${slot}.lock" ]; then
echo "$PID" > "$SEMAPHORE_DIR/slot_${slot}.lock"
echo "$slot"
return 0
fi
done
return 1
}
WAIT_TIME=0
while [ $WAIT_TIME -lt $T1 ]; do
SLOT=$(acquire_slot)
if [ -n "$SLOT" ]; then
log_message "Resource acquired (slot $SLOT)"
log_message "Using resource... (${T2}s)"
sleep $T2
rm -f "$SEMAPHORE_DIR/slot_${SLOT}.lock"
log_message "Resource released (slot $SLOT)"
exit 0
else
WAIT_TIME=$((WAIT_TIME + 1))

```

```

WAIT_TIME=$((WAIT_TIME + 1))
ACTIVE=$(count_active_processes)
log_message "Resource bussy Waiting... ($WAIT_TIME/${T1}s) [$ACTIVE active processes]"
sleep 1
fi
done
log_message "Timeout (${T1} sec)."
exit 1

```

Шаг 4: Запуск нескольких процессов

```

alexexi@fedora:~/work/os/lab14$ nano semaphore_multiple.sh
alexexi@fedora:~/work/os/lab14$ ./semaphore_multiple.sh 2 1
alexexi@fedora:~/work/os/lab14$ ./semaphore_multiple.sh 3 2
alexexi@fedora:~/work/os/lab14$ ./semaphore_multiple.sh 4 1

```

2.2. Скрипт для имитации команды man

Задание: Реализовать команду `man` с помощью командного файла. Командный файл должен получать имя команды в качестве аргумента и отображать его справочную страницу или сообщение об ошибке, если оно не существует.

Шаг 1: Создание скрипта

```
alexexi@fedora:~/work/os/lab14$ nano my_man.sh
alexexi@fedora:~/work/os/lab14$ chmod +x my_man.sh
```

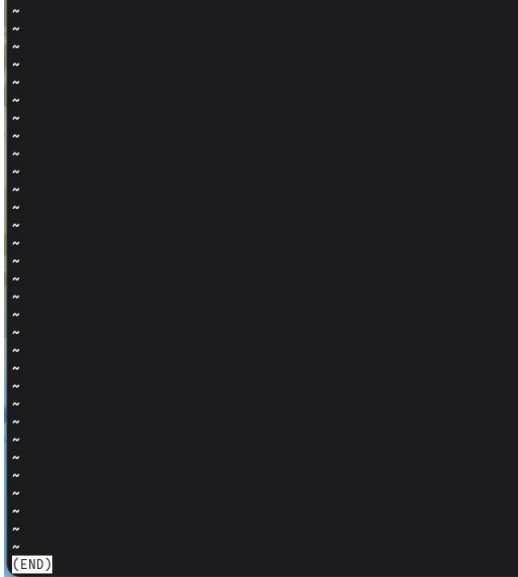
Код скрипта:

```
GNU nano 8.7.1 my_man.sh Modificado
#!/bin/bash
#script:my_man.sh
#description:Simulates the man command by displaying the manual page for the command
#Check for the existence of the passed argument

if [ $# -ne 1 ]; then
echo "Usage: $0 <command_name>"
echo "Example: $0 ls"
exit 1
fi
CMD="$1"
MAN_DIR="/usr/share/man/man1"
MAN_FILE="$MAN_DIR/${CMD}.1.gz"
#Check for the existence of the man file
if [ -f "$MAN_FILE" ]; then
echo "Manual '$CMD':"
echo "=====
#Displaying the content of the man file using gunzip -c and less
gunzip -c "$MAN_FILE" | less
elif [ -f "$MAN_DIR/${CMD}.1" ]; then
#Some manuals are not compressed
echo "Manual for '$CMD':"
echo "=====
#Displaying manual contents with gunzip -c and less
gunzip -c "$MAN_FILE" | less
elif [ -f "$MAN_DIR/${CMD}.1" ]; then
#Some manuals are not compressed
echo "Manual for '$CMD':"
echo "=====
less "$MAN_DIR/${CMD}.1"
else
#Search other sections
echo "Searching for manual in other sections..."
FOUND=$(find /usr/share/man -name "${CMD}.*.gz" 2>/dev/null | head -1)
if [ -n "$FOUND" ]; then
echo "Manual found in: $FOUND"
gunzip -c "$FOUND" | less
else
echo "Error: Manual for '{CMD}' not found"
echo "Try:man $CMD"
exit 1
fi
fi
```

Шаг 2: Просмотр содержимого каталога `man1`


```
alexei@fedora:~/work/os/lab14$ ./my_man.sh non-existen_command
```



2.3. Скрипт для генерации случайных последовательностей букв

Задание: использование встроенной переменной \$RANDOM, запись командного файла, генерация случайной последовательности букв. Случайного '\$RANDOM' генерирует псевдослучайные числа в диапазоне от 0 до 32767.

Шаг 1. Создайте сценарий.

```
alexei@fedora:~/work/os/lab14$ nano random_letters.sh  
alexei@fedora:~/work/os/lab14$ chmod +x random_letters.sh
```

Код скрипта:

```

GNU nano 8.7.1 random_letters.sh
#!/bin/bash
#Script:random_letters.sh
#Description: Generates a random sequence of letters using $RANDOM
#Function for generating a random letter
random_letter(){
#RANDOM generates numbers from 0 to 32767
#We take the remainder of 26 to get a number from 0 to 25
local idx=$((RANDOM % 26))
#Lowercase letters of the alphabet local letters=(a b c d e f g h i j k l m n o p q r
echo "${letters[$idx]}"
}
#Function for generating a sequence of a specific length
generate_sequence(){
local length=$1
local sequence=""
for ((i=0; i<length; i++)); do
sequence="${sequence}${(random_letter)}"
done
echo "$sequence"
}
#Function for generating a sequence using the array method (more efficient)
generate_sequence_array(){
local length=$1
local letters=(a b c d e f g h i j k l m n o p q r s t u v w x y z)
local result=""
result="${result}${letters[$((RANDOM % 26))]}"
done
echo "$result"
}
#Show options
echo "===Random Sequence Generator===
echo "1. Generate a sequence of 10 letters"
echo "2. Generate a sequence of a given length"
echo "3. Generate multiple sequence"
echo "4. Generate a sequence with uppercase and lowercase letters"
read -p "Select option (1-4):" OPTION case $OPTION in
1)
echo "Generated sequence: $(generate_sequence 10)";
2)
read -p "Sequence length:" LEN if [[ "$LEN" =~ ^[0-9]+$ ]] && [ "$LEN" -gt 0 ]; then
echo "Generated sequence: $(generate_sequence $LEN)"
else
echo "Error:Enter a valid number"
fi
;;
3)
read -p "Number of sequences to generate:" COUNT
read -p "Number of sequences to generate:" COUNT
read -p "Length of each sequence:" LEN
echo "Generating $COUNT sequences of length $LEN"
for ((j=1;j<=COUNT;j++)); do
echo "$j:$(generate_sequence $LEN)"
done
;;
4)
echo "Generated mixed sequence: $(generate_sequence_array 15 | tr 'a-z' 'a-zA-Z')"
*)
echo "Invalid parameter"
esac

```

Шаг 2: Расширенная версия с параметрами командной строки

bash

nano random_letters_cli.sh

chmod +x random_letters_cli.sh

Код скрипта (с аргументами командной строки):

```
GNU nano 8.7.1 random_letters_cli.sh Modificado
#!/bin/bash
#Script: random_letters_cli.sh
#Description: Generating random sequence with arguments CLI
usage(){
echo "Use: $0 [-l LENGTH] [-n Quantity] [-m]"
echo "  -l LENGTH LENGTH length of each sequence (default:10)"
echo "  -n Quantity Number of sequences to generate (default: 1)"
echo "  -m Include uppercase and lowercase letters"
echo "  -h Show this help"
exit 1
}
LENGTH=10
COUNT=1
MIXED=0
while getopts "l:n:mh" opt; do
case "$opt" in
1)LENGTH="$OPTARG";;
2)COUNT="$OPTARG" ;;
m)MIXED=1 ;;
h)usage ;;
*)usage ;;
esac
done
letters=(a b c d e f g h i j k l m n o p q r s t u v w x y z)
letters_mixed=(a b c d e f g h i j k l m n o p q r s t u v w x y z \ A B C D E F G H I J K L M N O P Q R S T U V W X Y Z)
generate_seq(){
local len=$1
local result=""
local result=""
if [ $MIXED -eq 1 ]; then
for ((i=0; i<len; i++)); do
result="${result}${letters_mixed[$((RANDOM % 52))]}"
done
else
for ((i=0; i<len; i++)); do
result="${result}${letters[$((RANDOM % 26))]}"
done
fi
echo "$result"
}
echo "====Generated of random sequence CLI===="
echo "Length: $LENGTH | Count: $COUNT | Mixed: $MIXED"
echo "===="
for ((j=1; j<=COUNT; j++)); do
echo "$j: $(generate_seq $LENGTH)"
done
```

Шаг 3: Запуск скриптов

```
alexei@fedora:~/work/os/lab14$ ./random_letters_cli.sh
====Generated of random sequence CLI====
Length: 10 | Count: 1 | Mixed: 0
====
52d: 1
52d: rzqztfhmad
```

```

alexei@fedora:~/work/os/lab14$ ./random_letters_cli.sh -l 15 -n 5
====Generated of random sequence CLI=====
Length: 15 | Count: 5 | Mixed: 0
=====
52d: 1
52d: lsdpugmrxtklcv
52d: 2
52d: rirddaocydroyfn
52d: 3
52d: pdpdtlylsheihya
52d: 4
52d: fvgzzuqzuqsavpe
52d: 5
52d: qnzxubgwjofrtju
alexei@fedora:~/work/os/lab14$ ./random_letters_cli.sh -l 20 -n 3 -m
====Generated of random sequence CLI=====
Length: 20 | Count: 3 | Mixed: 1
=====
52d: 1
52d: a[1]}a[39]}a[39]}a[18]}a[2]}a[18]}a[43]}a[7]}a[21]}a[16]}a[2]}a[41]}a[5]}a[49]}
a[45]}a[37]}a[6]}a[48]}a[8]}a[7]}
52d: 2
52d: a[48]}a[20]}a[50]}a[49]}a[33]}a[12]}a[9]}a[42]}a[15]}a[45]}a[31]}a[34]}a[42]}a[
9]}a[34]}a[38]}a[16]}a[17]}a[31]}a[21]}
52d: 3
52d: a[40]}a[36]}a[19]}a[34]}a[44]}a[17]}a[46]}a[16]}a[45]}a[25]}a[31]}a[27]}a[38]}a
[8]}a[39]}a[19]}a[27]}a[40]}a[5]}a[26]}

```

Контрольные вопросы

В этом разделе даются ответы на контрольные вопросы, касающиеся продвинутого программирования в Bash.

1. Найдите синтаксическую ошибку в следующей строке: `while [$1 != "exit"]`

Основная ошибка заключается в отсутствии пробелов вокруг скобок и операторов сравнения. В Bash правильный синтаксис для ``[(test)`` требует пробелов между элементами.

- Неправильно (как в вопросе):

```

bash
while [$1 != "exit"]

```

- Правильно:

```

bash

```

```
while [ "$1" != "exit" ]
```

Пояснение изменений:

- Пробел после `[` и перед `]`
- Пробелы вокруг оператора `!=`.
- Переменные должны быть заключены в двойные кавычки (`"\$1"`), чтобы избежать проблем, если аргумент содержит пробелы или пуст.

2. Необходимо ли подключать движок к сети?

В Bash конкатенация строк выполняется простым размещением переменных рядом друг с другом в одной строке.

Простой пример:

```
bash
str1="Hello"
str2="World"
result="$str1 $str2"
echo "$result" # Выводит: Hello World
```

Другие способы:

- Использование `+=`:

```
bash
text="Hello"
text+="World"
echo "$text" # Выводит: Hello World
```

- Использование `printf -v` (более эффективная версия для множества конкатенаций):

```
bash
printf -v result '%s' "$str1" "$str2"
```

```
echo "$result" # Выводит: Hello World
```

3. Найдите информацию о том, как использовать seq. Можно ли релизовать и ее функции при программировании в bash?

seq генерирует последовательности чисел. Пример: ``seq 1 5`` выводит 1 2 3 4 5.

Альтернативы в Bash:

1. С ``for`` и ``{start..end}`` (рекомендуется для фиксированных диапазонов):

```
`bash`  
`for i in {1..10}; do echo $i; done`
```

2. С ``while`` (для более сложных последовательностей):

```
`bash`  
`i=1`  
`while [ $i -le 10 ]; do echo $i; ((i++)); done`
```

3. С ``for ((...))`` (синтаксис, похожий на C):

```
`bash`  
`for ((i=1; i<=10; i++)); do echo $i; done`
```

4. Каков результат операции $\$(10/3)$?

Результат операции $\$(10/3)$ равен 3.

В Bash арифметические операции с $\$(...)$ работают с целыми числами (целочисленное деление). Результат отбрасывает десятичную часть, поэтому $10/3 = 3,333...$ становится 3.

5. Укажите кратко основные отличия командной оболочки zsh от bash.

- zsh (Z Shell) — более современная и продвинутая оболочка, чем Bash, с улучшенными функциями для интерактивного пользователя.
- Bash — это универсальная стандартная оболочка; она присутствует почти в каждой системе Linux, более стабильна и имеет лучшую поддержку скриптов.

Основные отличия:

Функции	Bash	Zsh
Автозавершение	Базовое	Интеллектуальное, с описаниями и параметрами
Совместимость	Универсальная (стандартная в Linux)	Не всегда устанавливается по умолчанию
Скрипты	Более стабильная и портативная	Менее портативная (скрипты могут давать сбои в других оболочках)
Настройка	Ограниченная	Высокая степень кастомизации (темы, плагины с Oh My Zsh)
Исправление ошибок	Не исправляет опечатки	Предлагает исправления для неправильно написанных команд

6. Проверьте правильность следующего синтаксиса: `for ((a=1; a <= LIMIT; a++))`

Да, синтаксис правильный (при условии, что `LIMIT` — предопределенная переменная). Это синтаксис циклов `for` в Bash в стиле C.

Полный и правильный пример:

```
bash
LIMIT=5
for ((a=1; a <= LIMIT; a++)); do
echo "Iteration $a"
done
```

7. Экономия времени и денег без программирования. Что вы думаете о том, чтобы попробовать это сделать? Какие недостатки?

Преимущества Bash:

- Прямая интеграция с операционной системой: Вы можете выполнять команды, управлять файлами и процессами без внешних библиотек.
- Простая автоматизация: Идеально подходит для административных задач, резервного копирования и рабочих процессов разработки (конвейеров).
- Установлен по умолчанию: Практически во всех системах Unix/Linux.

Недостатки:

- Неинтуитивный синтаксис: Легко допустить ошибки (например, забыть пробелы в []).
- Сложности со сложными операциями: Математика и расширенная обработка данных ограничены; все обрабатывается как текст.
- Менее структурирован: В отличие от языков, таких как Python или C, структуры данных ограничены (в старых версиях нет собственных объектов или ассоциативных массивов).
- Ограниченная переносимость: Скрипты, использующие специфические для Bash функции, могут не работать в других оболочках (например, sh или dash).

Выводы

В ходе выполнения лабораторной работы я изучил основы программирования в оболочке ОС UNIX и научился писать более сложные командные файлы с использованием логических управляющих конструкций и циклов. Я реализовал упрощённый механизм семафоров для синхронизации процессов с использованием файлов-блокировок, создал командный файл для имитации работы команды man с поиском справки в каталоге /usr/share/man/man1, а также разработал скрипт для генерации случайных последовательностей букв латинского алфавита с использованием встроенной переменной \$RANDOM. Все поставленные задачи выполнены, полученные навыки являются основой для автоматизации сложных задач и синхронизации процессов в операционной системе Linux.

Список литературы

- Cylab. (2026). Избегайте утечки учетных данных с помощью Pass: стандартного менеджера паролей для Unix. <https://cylab.be/blog/503/avoid-credential-leakage-with-pass-the-standard-unix-password-manager>
- Chezmoi. (б. д.). Настройка. <https://www.chezmoi.io/user-guide/setup/>
- Chezmoi. (б. д.). Что делает chezmoi?. <https://www.chezmoi.io/>
- Руководство по GNU Emacs. (б. д.). Хранилище паролей Unix. https://www.gnu.org/software/emacs/manual/html_node/auth/The-Unix-password-store.html
- Chezmoi. (б. д.). Справочная информация — шаблоны. <https://www.chezmoi.io/reference/templates/>
- Cylab. (2026). Избегайте утечки учетных данных с помощью Pass — стандартного менеджера паролей для Unix. <https://cylab.be/blog/503/avoid-credential-leakage-with-pass-the-standard-unix-password-manager>
- GoPass. (2026). gopass — чуть более продвинутый стандартный менеджер паролей для Unix для команд. <https://www.gopass.pw/>
- Password Store — Google Play. (2026). Password Store — приложение для Android. <https://play.google.com/store/apps/details?id=dev.msjarvis.aps>
- Хранилище паролей. (2026). Хранилище паролей: стандартный менеджер паролей для Unix. <https://www.passwordstore.org/>
- Проект Fedora. (2026). Руководство по системному администрированию DNF. https://docs.fedoraproject.org/en-US/fedora/latest/system-administrators-guide/package-management/DNF/Command_Reference/
- Chezmoi. (б. д.). Краткое руководство. <https://www.chezmoi.io/quick-start/>
- Интерфейс командной строки GitHub. (2026). Руководство по интерфейсу командной строки GitHub. <https://cli.github.com/manual/>
- *GPG. (2026). Документация по GNU Privacy Guard. <https://gnupg.org/documentation/>

- How-To Geek. (2021). Как создать семафор в Bash.
<https://www.howtogeek.com/devops/how-to-create-a-semaphore-in-bash/>
- Baeldung. (2023). Используйте \$RANDOM для генерации случайной строки.
<https://www.baeldung-cn.com/linux/generate-random-string-using-random>
- CloudComputing-Insider. (2022). Использование случайных строк в Bash.
<https://www.cloudcomputing-insider.de/zufallsquellen-in-der-bash-nutzen-a-b7c25dcc121b0161f7501eebb98d45db/>
- Unix & Linux Stack Exchange. (2020). Пользовательская команда man.
<https://unix.stackexchange.com/questions/>
- Baeldung. (2023). Использование \$RANDOM для генерации случайной строки.
<https://www.baeldung-cn.com/linux/generate-random-string-using-random>
- GNU Savannah. (2022). Неожиданный пробел в условном операторе.
<https://savannah.gnu.org/support/index.php?110745>
- Refine. (2024). Zsh и Bash. <https://refine.dev/blog/zsh-vs-bash/>